

Unit - IV Schema Refinement (Normalization)

* Normalization (Schema Refinement):

Schema Refinement or normalization is used to eliminate the redundancy. Based on decomposition techniques. Here redundancy means duplication of data, i.e. same data exists more than one time. It may be useful to retrieve the lost information when disk failure is occurred. But redundancy may leads to several problems.

Here decomposition means division of one table into two tables.

* Need of normalization:

- ⇒ Data redundancy
- ⇒ Insertion anomalies
- ⇒ Deletion anomalies
- ⇒ updation anomalies

⇒ Data Redundancy: Duplication of data.

Ex: some information is stored repeatedly.

⇒ Insertion anomalies: some times it may not be possible to insert certain information unless and until some other information is needed to insert with it.
i.e., we need to insert required data as well as with not required data (related / non-related)

→ Deletion anomalies: sometimes it may not be possible to delete certain information unless and until some other information is needed to delete with it. i.e., we need to delete required data, as well as with not required data.

→ Update anomalies: sometimes it may not be possible to update the repeated information (duplicate data) because some inconsistency is created unless unless all the duplicated copies are updated.

Student course table

SID	Sname	CID	Cname	Cfee
S1	A	C1	C	5000
S2	A	C1	C	5000
S1	A	C2	C	10000
S3	B	C2	C	10000
S3	B	C2	JAVA	15000

Redundancy (circles around S1, S2, S1, S3, S3 and C1, C1, C2, C2, C2)

Redundance (arrow pointing to the table)

Redundance storage: Here SID S1 Sname A
 SID S3 Sname B
 CID C1 Cname C

CID C2 Cname C

are redundantly stored in the student course table
 This may leads to some inconsistency.

Insertion anomalies:

SID	sname	cid	cname	cfee
S1	A	C1	C	5000
S2	A	C1	C	5000
S1	A	C2	C	10000
S3	B	C2	C	10000
S3	B	C2	JAVA	15000

Now we are
inserting course C3 details.

It is possible until and unless we insert the student details of course C3

SID	sname	cid	cname	cfee
S1	A	C1	C	5000
S2	A	C1	C	5000
S1	A	C2	C	10000
S3	B	C2	C	10000
S3	B	C2	JAVA	15000
		C3	SAL	20000

Insertion
anomaly

Student details needed to be
insert to insert course 3 details.

⇒ deletion anomalies: If we want to delete student S₂ details we need to delete along with his course 1 details

SID	Sname	CID	Cname	Cfee
S1	A	C1	C	5000
S2	A	C1	C	5000
S1	A	C2	C	10000
S3	B	C2	C	10000
S3	B	C2	JAVA	15000

deletion anomaly.

↳ To delete student retains need to delete course 1 details of S₂

⇒ updatation anomalies: If we want to update a course fees by 5000. so there may be repeated updatation because of redundancy. Now C1 Id / C1 course ID

C1 C is updated 10,000 2 times.

C2 C is also updated to 15000 (when we consider course only)

Repeated updatation is performed. due to updatation anomaly.

* Fun

that will

the for

del

SID	Sname	CID	cname	Cfee	
S1	A	C1	C	5000	⇒ S1 A C1 C 10000
S2	A	C1	C	5000	S2 A C1 C 10000
S1	A	C2	C	10000	} updated to 15000 inconsistency occurred.
S3	B	C2	C	10000	
S3	B	C2	Java	15000	

* Functional dependency:- (FD)

Functional dependency (FD) is a relationship that exists when one attribute uniquely determines another attribute.

⇒ Functional dependency is a kind of integrity constraint that can identify the schema with redundant storage problem and used for schema refinement.

⇒ Let R be a relation and x and y are the two non-empty sets of attributes in R . Therefore $FD: x \rightarrow y$

Functional
dependency

x is a
attribute or
set of attributes

y is a attribute
or set of attributes

We can say that x functionally determines y

* Rules of functional dependency or closures of functional dependency:

Rule:-1 Transitivity

If $x \rightarrow y$, $y \rightarrow z$ then $x \rightarrow z$

Rule:-2 Union

If $x \rightarrow y$, $x \rightarrow z$ then $x \rightarrow yz$

Rule:-3 decomposition

If $x \rightarrow yz$ then $x \rightarrow y$, $x \rightarrow z$

Rule:-4 Augmentation

If $x \rightarrow y$ then $xz \rightarrow yz$ for any z

Rule:-5 Pseudo transitivity

If $x \rightarrow y$ and $yz \rightarrow p$ then $xz \rightarrow p$

Rule:-6 composition rule

If $A \rightarrow B$ and $x \rightarrow y$ then $Ax \rightarrow By$

Rule:-7 Reflexivity

* If $x \supseteq y$ then $x \rightarrow y$

* $A \rightarrow A$ and $A \rightarrow B$

$$A \supseteq B$$

* Example:-

Sid	Sname	Sage
1	XX	19
2	YY	20
3	ZZ	21

In the student table the attributes are Sid, Sname, Sage

Here Sid attribute is used to uniquely identify the Sname attribute because if you know the student id no or Id we can easily tell the student name associated with that number. Therefore Sid $\rightarrow$ Sname

FD: Sid $\xrightarrow{\text{determines}}$ Sname

This is called functional dependency

Here Sname is functionally dependent on Sid

Here Sid is functionally determines Sname

FD is represented by the symbol " $\rightarrow$ " (arrow)

Here left hand side of attributes determines the values of right hand side attributes.

Functional dependency is an important part of relational database. and are used in Normalization techniques

* Advantages of functional dependency:

- 1) FD avoids the data redundancy
- 2) It maintains the quality in the database
- 3) It expresses the facts about database design
- 4) It helps in identifying the identifying the bad design

A	B	C
1	2	3
1	2	3
2	3	4
2	4	4
3	4	7

$A \rightarrow B, B \rightarrow C, AB \rightarrow C$

must satisfy

* Normal Forms:

There are different types in Normal forms to

normalize the data. They are

- 1) First Normal form (1NF)
- 2) Second Normal form (2NF)
- 3) Third Normal form (3NF)
- 4) Boyce Codd Normal form (BCNF)
- 5) Fourth Normal form (4NF)
- 6) Fifth Normal form (5NF)

First Normal Form (1NF)

According to EF. odd a relation will be ~~enough~~ ^{In 1NF}
Each attribute contains only atomic value (single value)

1NF states that,

An attribute of a relation does not contain multiple values. Values are stored in same domain.

Example:-

Student

Sid	Sname	Scourse
1	dai	PYP, C
2	ram	SQL
3	deetha	DS, JAVA

NOT in 1NF

After
normali-
zation.

Sid	Sname	Scourse
1	dai	PYP
1	dai	PYP C
2	ram	SQL
3	deetha	DS
3	deetha	JAVA

in 1NF

Second Normal Form (2NF)

A relation must in 2NF must satisfy following conditions:

- It is a relation must be in 1NF
- It must satisfies the rule of there is no partial dependency i.e. all non-prime attributes should dependent on prime attributes

Prime attribute :- Prime attributes are key attributes which are used to form a candidate key are called prime attributes.

Non-Prime attributes :- Non-key attributes are called non-prime attributes. i.e. non-prime attributes do not form candidate key.

AP	NP	P	NP
Sid	Sname	Pid	Pname

Sid $\rightarrow$ Sname Pname

Pid $\rightarrow$ Pname

Sid	Sname	Pid	Project name
S1	da	P1	mini project
S2	nam	P2	main project
S3	deetha	P3	CSP project

Here primary and prime attributes are Sid, Pid

non prime attributes are Sname, Projectname

Sid $\rightarrow$ Sname

Pid $\rightarrow$ Projectname

Sid	Sname
S1	Sai
S2	nam
S3	deethai

Sid → Sname

Pid	Project name
P1	mini project
P2	main project
P3	CSP project

Pid → Project name

Example:-

Sid	Sname	bid	bname
4201	X	42	AIML
4209	Y	42	AIML
4301	A	43	AI
4405	B	44	DS
0504	C	05	CSE

bid → Sname, bname

bid → bname

Sid	Sname	bname
4201	X	AIML
4209	Y	AIML
4301	A	AI
4405	B	DS
0504	C	CSE

Sid → Sname, bname

bid	bname
42	AIML
42	AIML
43	AI
44	DS
05	CSE

bid → bname

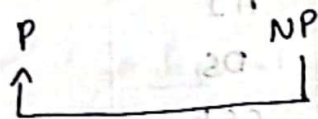
Step:-

Sid	Sname	Saddress	Subid	Subname
1	dai	KKD	S1	DBMS
2	dei	KKD	S2	OS
3	Ram	RJY	S3	JAVA
4	dita	TTD	S4	PYP

Here prime attributes are Sid, Subid

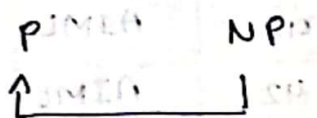
non-prime attributes are Sname, Saddress, Subname.

Sid $\rightarrow$ { Sname, Saddress }



non-prime attributes Sname, Saddress fully dependent on Sid

Subid $\rightarrow$ { Subname }



Nonprime attribute Subname fully dependent on Subid

After decomposition the schema is. Sid $\rightarrow$ Sname, Saddress

sid	Sname	Saddress
1	dai	KKD
2	dai	KKD
3	Ram	RJY
4	dita	TTD

subid	subname
S1	DBMS
S2	OS
S3	Java
S4	PYP

sid $\rightarrow$ Sname, Saddress.

subid $\rightarrow$ subname

now relation is in 2NF

Example :- 3

sid Sname brandid branchname

sid $\rightarrow$ Sname

brandid $\rightarrow$ branchname

Example :- 4

sid Sname Gender

After decomposition

sid $\rightarrow$ Sname

sid $\rightarrow$ Gender.

Example :- 5

customerid. customername productid product name

Quantity address

customer id	customer name	product id	product name	quantity	address
-------------	---------------	------------	--------------	----------	---------

customer id $\rightarrow$ customer name, address

$\Rightarrow$ 3NF:

3NF means Third Normal Form. A relation in 3NF must follow the following condition.

$\Rightarrow$ A relation must be in 2NF

$\Rightarrow$ It must satisfy the rule of no transitive dependency or non-trivial dependency.

Transitive dependency:-

If $x \rightarrow y$

$y \rightarrow z$ then $x \rightarrow z$

Example:-

Sid	sname	Pincode	address
1	Rai	123	KKD
2	Ram	456	RJY
3	Rita	789	TTD

$Sid \rightarrow \{sname, address\}$

$Pincode \rightarrow \{address\}$

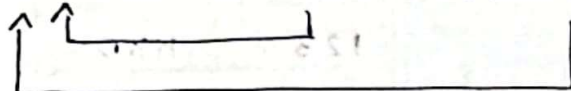
in e
since
non
acc
depe

In our new schema prime attributes are sid, pincode.

non-prime attributes are sname, address.

According to 2NF Every non-prime attribute should be dependent on prime attribute

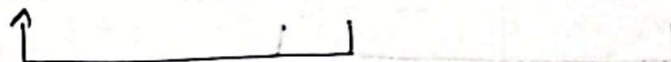
sid $\rightarrow$ {sname, address}



non prime depends on prime

According to 2NF

Pincode $\rightarrow$ {address}



non prime depends on

Prime according to 2NF

According to 3NF There is no transitive dependency.

sid $\xrightarrow{\text{determines}}$ {sname, address}

not determines

Here there is no transitive dependency because

sid $\xrightarrow{\text{determines}}$ sname

sname $\xrightarrow{\text{not determines}}$ address

So schema is in 3NF

According to 3NF there is no transitive dependency.

Pincode $\xrightarrow{\text{determines}}$ address

So schema is in 3NF

After the Normalization, the relation decomposed as follows.

Sid	Sname	address
1	dai	KKD
2	Ram	RJY
3	Sila	TTD

Sid $\rightarrow$ Sname, address

Pincode	address
123	KKD
456	RJY
789	TTD

Pincode $\rightarrow$ address

Now the relation is in 3NF

* Example 2

Sid	Sname	address	DOB	age
1	X	TTD	01/01/05	18
2	Y	KKD	05/06/06	17
3	Z	RJY	09/10/05	18

Here prime attributes are Sid, DOB.

non-prime attributes are Sname, address, age

According to 2NF there is no partial dependency.

According to 3NF there is no transitive dependency.

Now the schema will be,

$Sid \rightarrow \{Sname, address\}$

$DOB \rightarrow \{age\}$

Sid	Sname	address	DOB	age
1	X	TTD	01/01/05	18
2	Y	KKD	05/06/06	17
3	Z	RJY	09/10/05	18

$Sid \xrightarrow{\text{determines}} Sname, address$ $DOB \rightarrow age$

$Sid \xrightarrow{\text{determines}} Sname$ $DOB \xrightarrow{\text{determines}} age$

$Sname \rightarrow address$

not
determines

It is in 3NF

So it is in 3NF

⇒ BCNF:-

BCNF means Boyce Codd Normal form. It is an extension of 3NF. If a relation is in BCNF that should be followed 3NF. It should follow the following rule

⇒ For any dependency $a \rightarrow b$; a should be a super key

Super key:- Super key is a set of keys those may be candidate keys or Primary keys.

Ex:- For employee database Superkey = { employee id, license number, passport number }.

studentid	studentname	branchid	branchname
1	Ani	05	CSE
2	Ram	42	AIML
3	Wishna	44	DS
4	Sita	05	CSE

Here branchid is not the superkey because there is duplication of branchid = 05.

superkey does not contain duplicate values.

According to 1NF we need to write atomic value it is satisfied.

According to 2NF Here Prime attributes are studentid, branchid

non-prime attributes are student name, branch name

According to 2NF there is no partial dependency.

According to 3NF the schema will be

studentid $\rightarrow$ { student name }

Here student id is the key attribute & is a

Super Key

StudentId	StudentName	branchId	branchName
1	dai	05	CSE
2	Ram	42	AIML
3	Krishna	44	DS
4	dita		

relation
in BCNF

branchId $\rightarrow$ branchName.

Here, branchId is the super
key.

4NF:

4NF means fourth normal form. A relation in 4NF must satisfy BCNF.

$\Rightarrow$ It should follow the rule of no multivalued dependency

Multivalued dependency: For a dependency $a \twoheadrightarrow b$

for a single value of a multiple values of b exist then the table may have multivalued dependency.

$\Rightarrow$ A table should have at least 3 columns for it to have multivalued dependency.

$\Rightarrow$ For a relation $R(a, b, c)$ if there is a multivalued dependency between a & b .

then b & c should be independent with each other

$\Rightarrow$ If all these conditions are true for any relation

then it said to have multivalued dependency.

student id	student name	course
101	Ankit	Python
102	Karthik	Java
103	Krishna	R Programmi
104	Ankit	Java
105	Akash	PHP Python

course depends on student id

NOT in 4NF

student name depends on student id

The above relation the independent attributes are student name and course. Both depends on student id. In this case two attributes name & course are dependent on single attribute student id.

This is called multivalued dependency.

After decomposition the relations are

student id	student name
101	Ankit
102	Karthik
103	Krishna
105	Akash

student id	course
101	Python
102	Java
103	Rprogra
101	Java
105	Php

student id → student name

single

single

student id → course

single

single

Examp

stu
10
10
10
10
10
10

stu

NOT

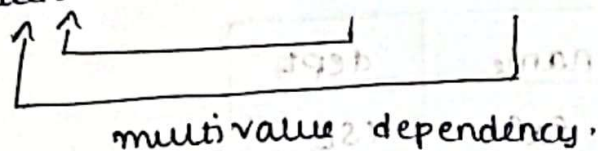
NOI

Aft

Example: 2

Student Id	Subject	Activity
100	music	Swimming
100	accounting	Swimming
100	music	Tennis
100	accounting	Tennis
150	Maths	Jogging

Student Id $\rightarrow$ { Subject, Activity }.



NOT in 4NF, so decompose the relation.

Now, the schema is.

After decomposition the relation is

Student Id $\rightarrow$ Subject

StudentId	Subject
100	music
100	accounting
150	maths

Student Id $\rightarrow$ Activity.

StudentId	Activity
100	Swimming
100	Tennis
150	Jogging

Now the relation is in 4NF

⇒ 5NF:-

when a relation is in 5NF it should satisfy the following condition.

- ⇒ The relation must be in 4NF
- ⇒ The relation must satisfy the rule lossless join dependency i.e. we need to decompose the table after that rejoin the decomposed tables. then we get the original table, i.e. no loss of the data.

Example:-

Round	name	dept
111	dai	CSE
222	dai	ECE

Round $\rightarrow$ name

Round	name
111	dai
222	dai

Round $\rightarrow$ dept

Round	dept
111	CSE
222	ECE

⋈

Join the decomposed table.

Round	name	dept
111	dai	CSE
222	dai	ECE

Round $\rightarrow$ { name, dept }

* Example

* Example:-2

S	P	D
S ₁	P ₁	d ₁
S ₂	P ₂	d ₂
S ₃	P ₁	d ₂

After decomposition:

$S \rightarrow P$

S	P
S ₁	P ₁
S ₂	P ₂
S ₃	P ₁

$S \rightarrow D$

S	D
S ₁	d ₁
S ₂	d ₂
S ₃	d ₂

⋈

S	P	D
S ₁	P ₁	d ₁
S ₂	P ₂	d ₂
S ₃	P ₁	d ₂

$S \rightarrow \{P, D\}$

lossless join and decomposition

lossless join and dependency preserving decomposition

Exa

decomposition: decomposition of a relation means division of one relation into two relations.

decomposition helps in elimination of the bad database design problems.

Ex: Example, redundancy, inconsistency and anomalies.

lossless join decomposition:

When a Relation R into R_1 and R_2 is lossless when join of R_1 and R_2 produce the same relation R . This is called lossless join decomposition and also called as non-additive decomposition.

Let relation R

Subrelations R_1, R_2 .

Lossless decomposition $\Rightarrow R_1 \bowtie R_2 = R$

$R_{\text{Join}} = R_{\text{Actual}}$

Lossy decomposition $\Rightarrow R_1 \bowtie R_2 \supset R$

Example:-

Round	name	dept
111	Sai	CSE
222	sai	ECE

Round $\rightarrow$ name

Round	name
111	sai
222	sai

Round $\rightarrow$ dept

Round	dept
111	CSE
222	ECE

Round	name	dept
111	Sai	CSE
222	sai	ECE

$R_1 \bowtie R_2 = R$ lossless

* File Organisation :- File Organisation contains various File Organisation used to describe the where in which the records are stored in terms of blocks.

Simple file Organisation tells the way how the data is stored in a file (and the methods to access the data).

→ file is a collection of records.

Objective of the file Organization or features :- Record should be accessed as fast as possible insert, delete, update, transactions on records are quickly and easily.

Records are stored in an efficient manner with minimum cost. Duplicate are not allowed.

Types of file Organisations :-

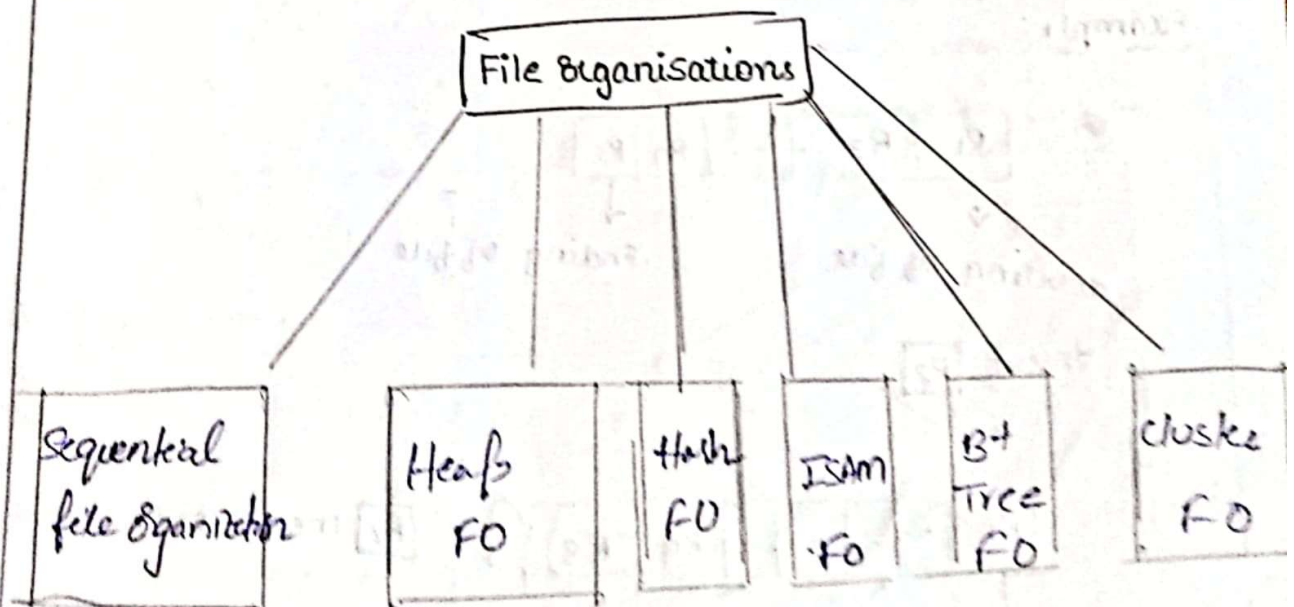


fig file organization.

* File Organisation :- File Organisation contains various File Organisation used to describe the where in which the records are stored in terms of blocks.

Simple file Organisation tells the way how the data is stored in a file (and the methods to access the data).

→ file is a collection of records.

Objective of the file Organization or features :- Record should be accessed as fast as possible insert, delete, update, transactions on records are quickly and easily.

Records are stored in a efficient manner with minimum cost. Duplicate are not allowed.

Types of file Organisations :-

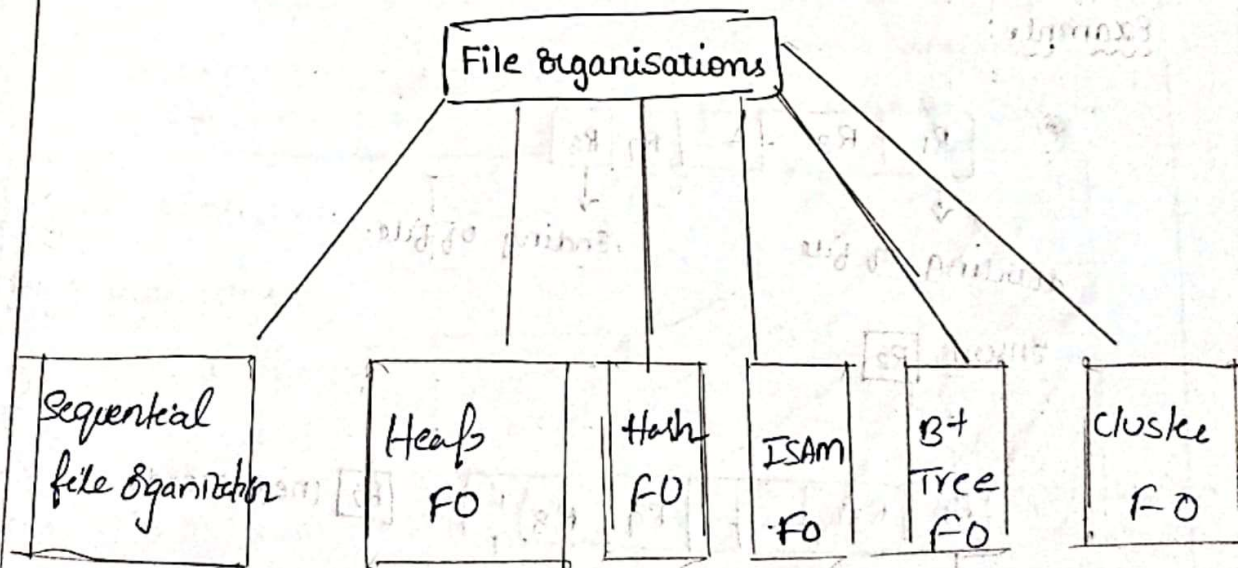
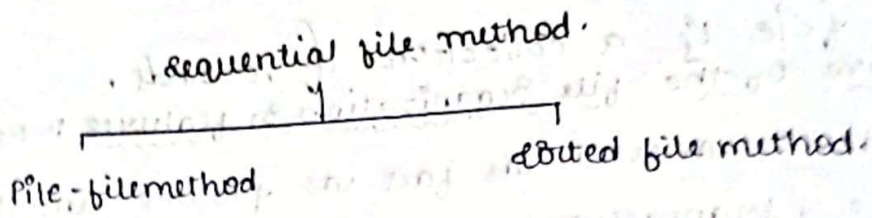


fig file organization.

→ Sequential file organisation: In this method files are divided in a sequence manner. Sequential file organisation is divided into two types.

- 1) File file method.
- 2) deque added file method.

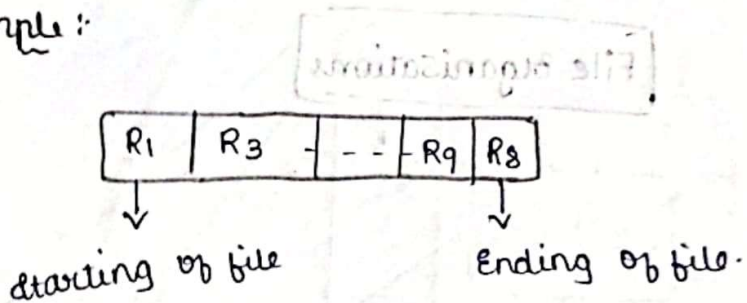


* File-file method: In this method we store the records in sequence one after another.

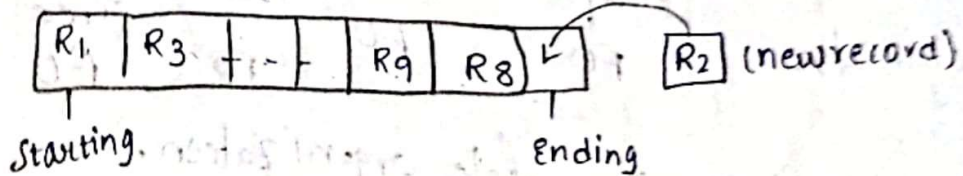
we will insert the record in sequence order in which they were inserted in a table.

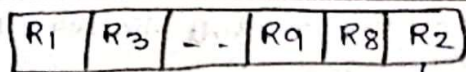
to delete and update records we need to search in the memory blocks. when the element is found we will delete.

Example:



Insert R_2





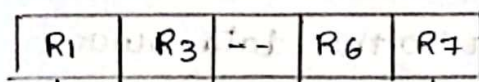
starting

ending

* Sorted file method: In this method new record is inserted always at the end of the file then it will sort the records in sequence in ascending or descending order based on primary key or any key.

TO update any record it will update the record and sort the file now updated record will be in right place of sorted file.

Example:-

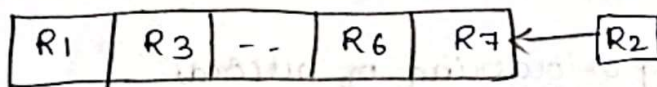


starting of a file



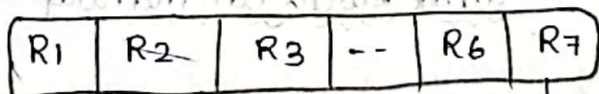
ending of a file

insert R2



R2 is inserted at end of the file.

After sorting



Start

End

R2 is inserted in b/w R1 and R3 after sorted.

Pros and cons:-

Pros:-

We can easily insert the records because of simple design design.

2) It is used when most of the records have to be accessed
Ex: Student Grade calculation. (All subject marks need to be accessed)

Cons:

1) We need to access the records sequentially only. So it takes more time.

2) To add every time it will take more time and space.

* Heap file Organization :- It is a simplest file organization

It works with data blocks

3) When data block is full new record is inserted in other block which is ^{not a} very next to the data block.

4) It can select any data block to store.

5) In Heap file organization records are inserted at the file end only. So

6) There is no adding or ordering of record

7) Heap file is also known as an unordered file

data records

data blocks in memory

R1
R3
R6
R4
R5

736
538

data block-1

637
273

data block-2

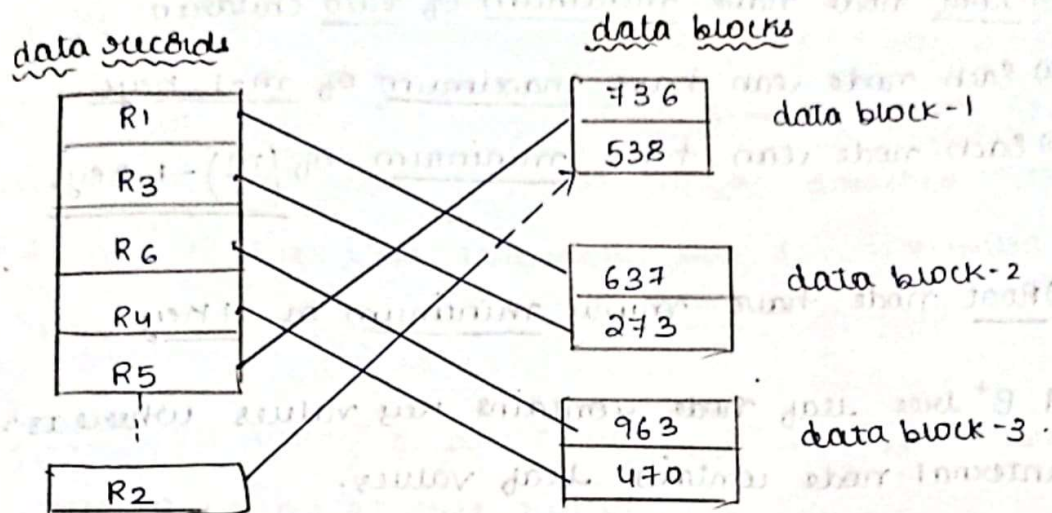
963
470

data block-3

essed
need

Here file contains 5 records R_1, R_3, R_6, R_4, R_5 in a heap

insert new record R_2 . Here data block-3 is full then it will be insert in data block-1 based on heap file organization.



* B⁺ tree file organization:

B⁺ tree is called as Balanced M-way search tree

B⁺ tree follows multi-level indexing. All the leaf nodes are at the same level. The leaf nodes are linked with one another using a linked list.

Every leaf node in B⁺ tree having equal distance from its root node. B⁺ tree is a order of n children

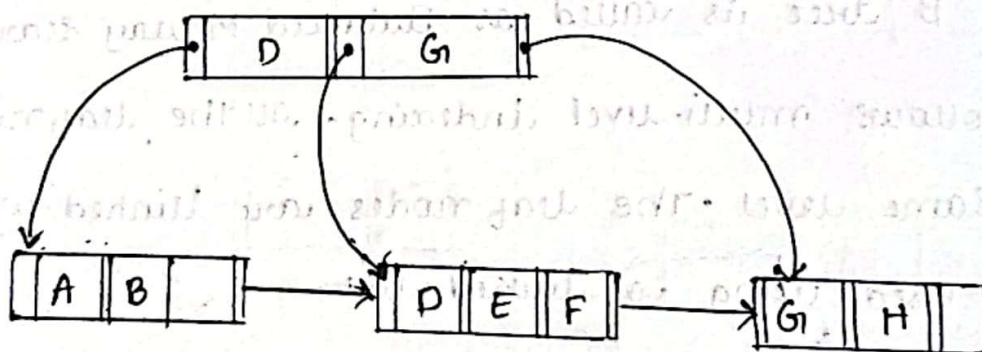
Here n is fixed

Rules of a B⁺ tree :-

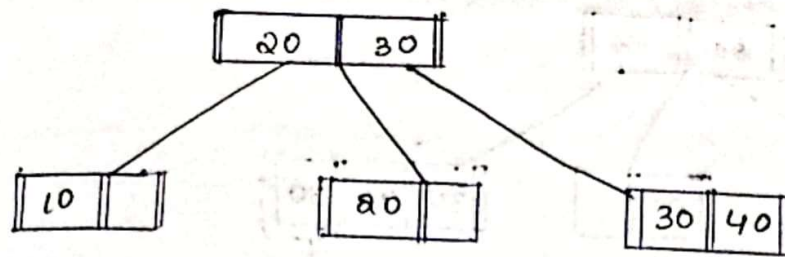
- 1) Each node have maximum of m children
leaf node have no children
- 2) Each node can have minimum of m/2 children
- 3) root node have minimum of two children
- 4) Each node can have maximum of m-1 keys
- 5) Each node can have minimum of $\lceil \frac{m}{2} \rceil - 1$ keys
- 6) Root node have minimum of 1 key

A B⁺ tree leaf node contains key values whereas internal node contain leaf values.

Structure of a B⁺ file tree file organization :



Example :-



TO search the record 10 in the B⁺ tree First we will search with root node and our search element is less than root node then it will be directed to the first key of the first leaf node. Now search element 10 is found.

* Insertion: TO insert & delete in a B⁺ tree file organization we need to follow B⁺ tree rules.

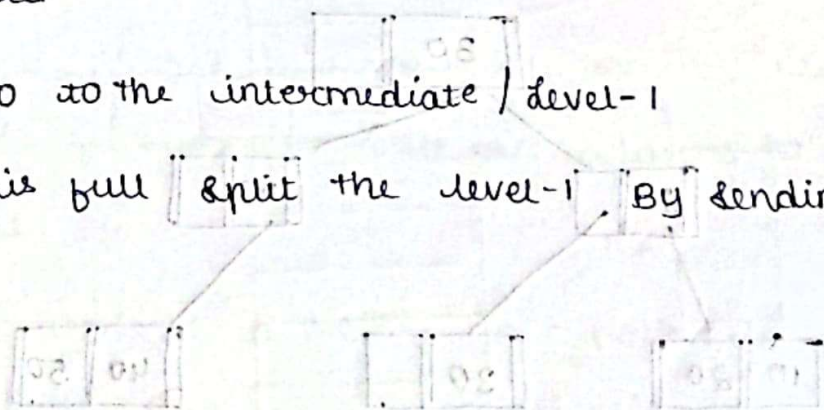
Ex:-

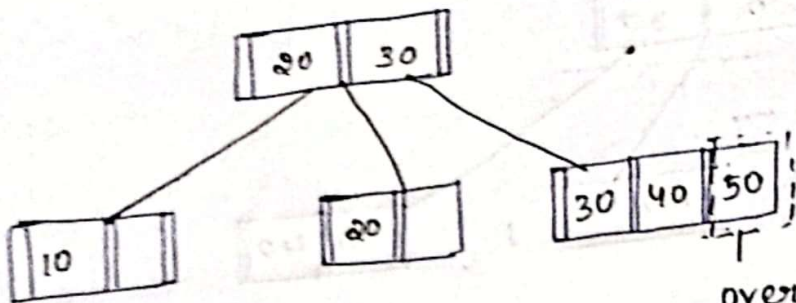
TO insert Key 50 in the above B⁺ tree

TO insert key 50 in last leaf node is already filled so split that node

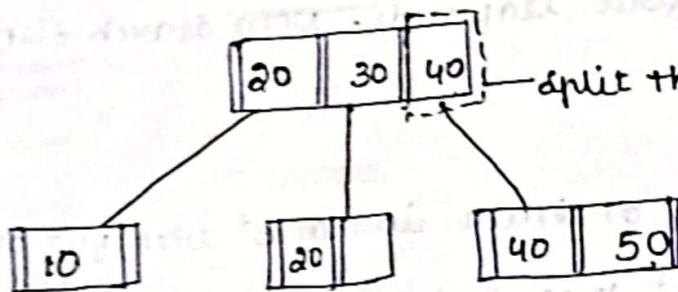
⇒ Now push 40 to the intermediate / level-1

⇒ Now level-1 is full split the level-1 By sending 30 to root level

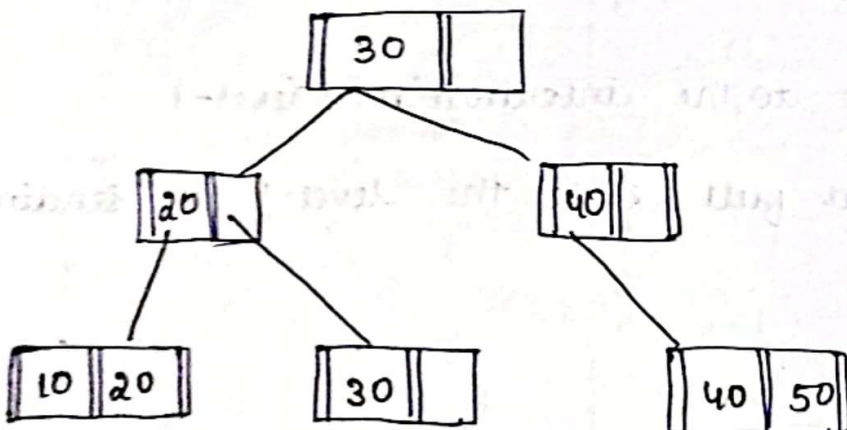




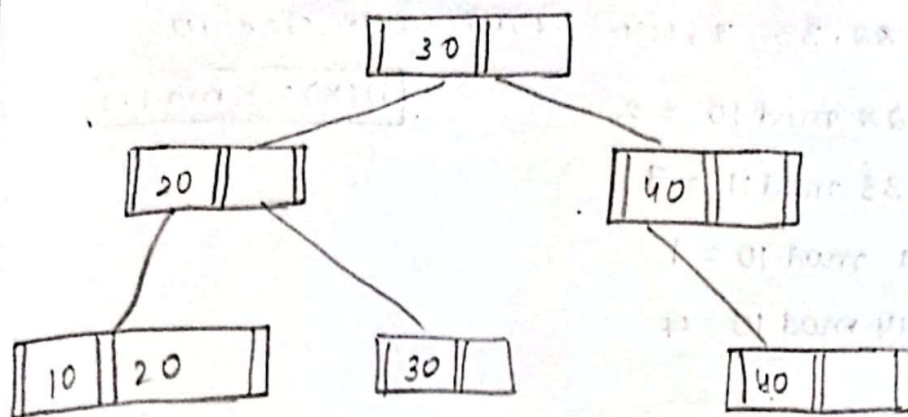
overflow split the node



split the node



deletion:-



Hash file Organization: Hash file Organisation is the Process of mapping large data values into small table using hash function.

⇒ Here hash key is used to map the value by using Hash address.

⇒ Hash function is used to search the value by using Hash key

⇒ To insert a new record in a hash file, we will use hash function this function generates an address to insert our value.

⇒ To delete and update a record in a Hash file Organisation same process is an application of an insertion process.

⇒ To search a record we can search by use a key

directly no need to search all records.

Example:- 22, 33, 11, 44.

Hash table size 10

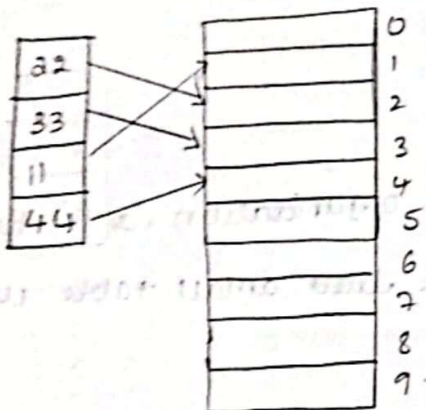
$$h(K) = K \bmod 10$$

$$h(22) = 22 \bmod 10 = 2$$

$$h(33) = 33 \bmod 10 = 3$$

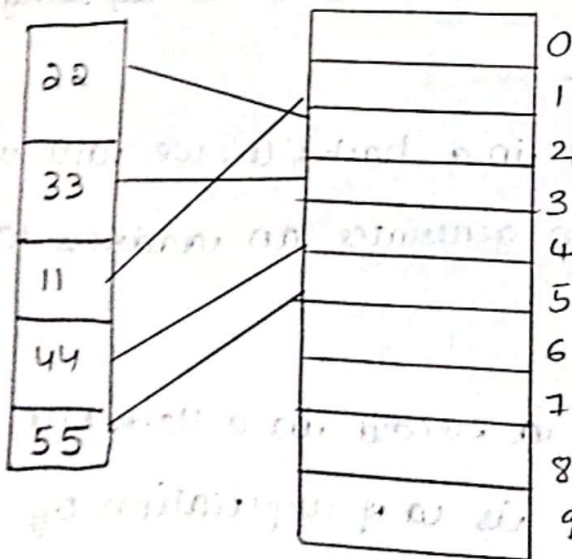
$$h(11) = 11 \bmod 10 = 1$$

$$h(44) = 44 \bmod 10 = 4$$



insert a record 5

$h(55) = 55 \% 10 = 5$ it generates hash address 5.



Types

Here

1) 220

stat

tr

0

w

th

is

or

Types of hashing:-

Here are two types of hashing

- 1) static hashing
- 2) dynamic hashing

static hashing:

there are two types of static hashing techniques

- 1) open hashing
- 2) closed hashing

when a hash function generates to insert a key if that address is already filled with data we can't insert our record in that address.

By using linear probing we will insert our record in the next available bucket.

	Records	
10	Record 10	0
12		0
33	11	1
11	2	2
44	33	3
55	5544	4
20	2055	5
	20	6
		7
		8
		9

addressing is

already filled with

a key 10 so insert

it in another

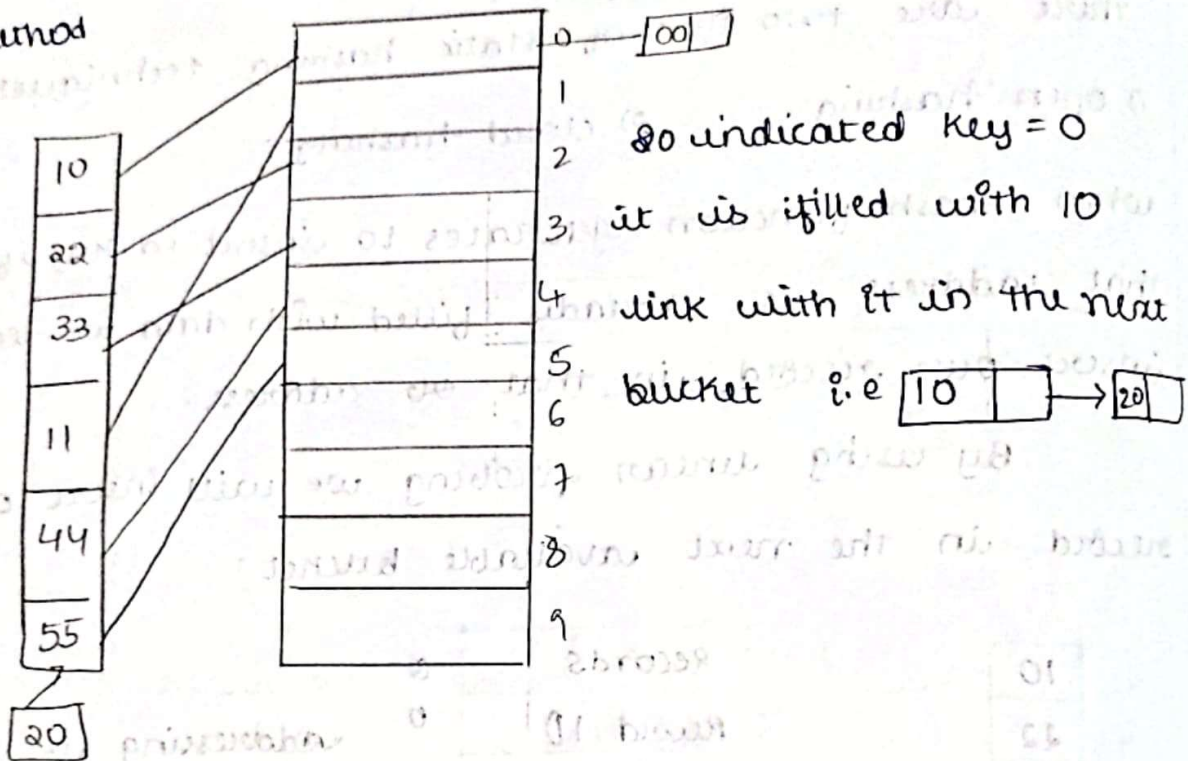
available free space

i.e 6

closed hashing :-

In closed hashing when bucket is full new bucket is allowed allocated to insert linked to previous bucket which is full. This is called overflow chaining method.

method



Insert record 20 it indicated key=0 which is already filled with a new bucket is linked with it to be 20 in that bucket.

Based on the hash key and hash function we will insert a new record delete a record & update a record.

Dynamic hashing: In dynamic hashing hash table size may change dynamically i.e. buckets can grow or shrink as the records increase (or) decrease. This method is also known as extendable hashing.

Key	hash address
1	11010
2	00000
3	11110
4	00000
5	01001
6	10101
7	10111

Bucket size = 2

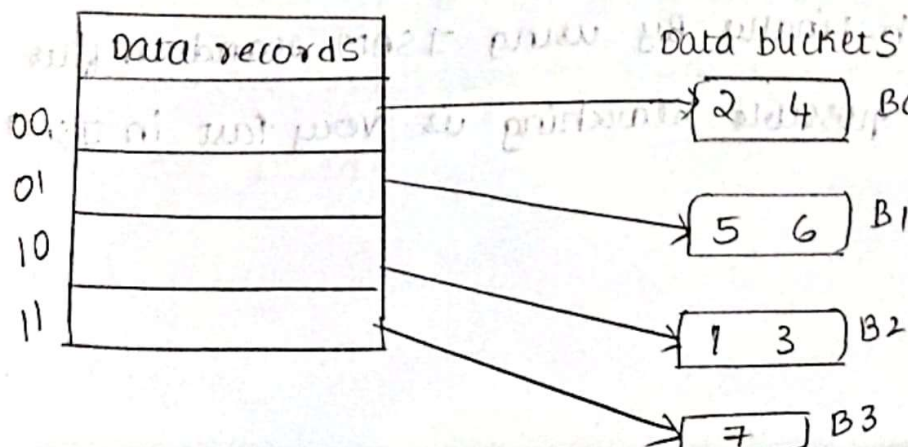
Here last two bits of 2 and 4 are 00. So these records are stored in bucket B0.

Last two bits of 5 and 6 are 01. These are stored in Bucket B1.

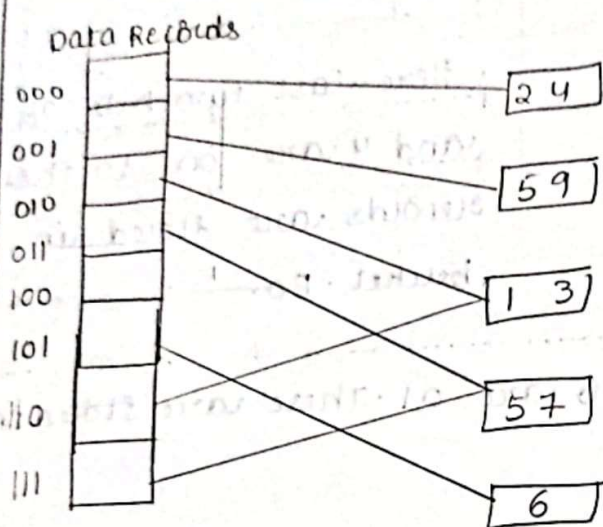
Last two bits of 1 and 3 are 10. These records are stored in Bucket B2.

Last two bits of 7 are 11. These records are stored in Bucket B3.

The dynamic allocation with hash function is as follows.



Insert the record 9 which is having hash address 10001. Last 2 digits of 9 is 01 so it is stored in the Bucket B₁ which is already full now B₁ Bucket is will be split. Now we will take the hash index upto 3 bits. The dynamic allocation with hash function as follows.



ISAM file Organisation:-

In ISAM method will access the files sequentially. In this method we store the records by using primary key.

An index value is generated for each primary key & mapped with the record. This index contains address of the record with finally. By using ISAM random files access also possible searching is very fast in ISAM.

Index (Partial index)

Bunny	5
chintu	10
	15

(5-9)

(10-14)

#	name	branch
0		
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		

INDEXING:-

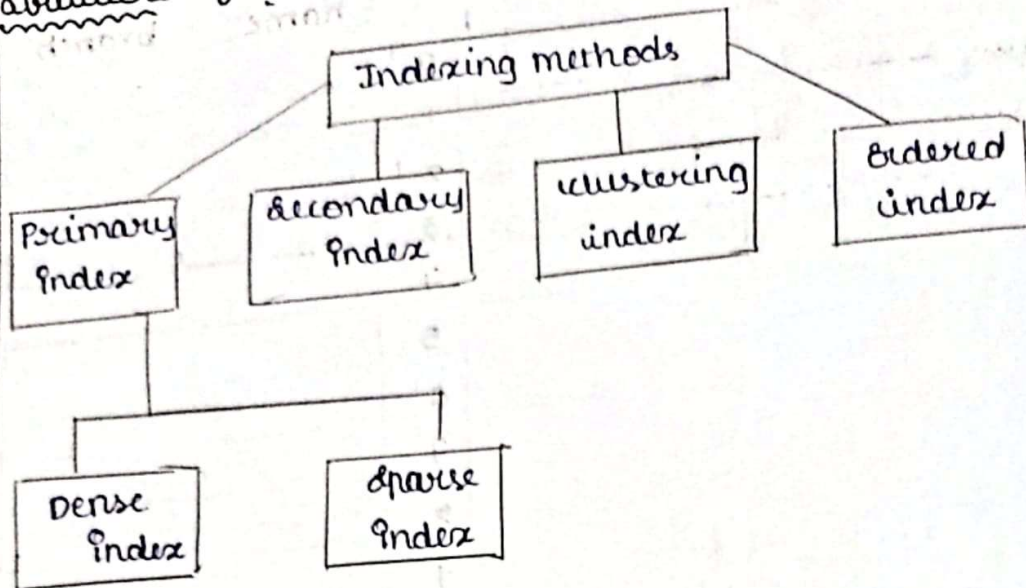
An index is data structure which allows to search and retrieve the records from a file efficiently & quickly.

Structure of an index:

Here in the structure which allows First column is a search key that contain the Primary Key Values which are used access corresponding in unique manner.

Second column in the structure contains set of pointers that holds the addresses of database blocks where the values are stored.

Structure of types of indexing methods:



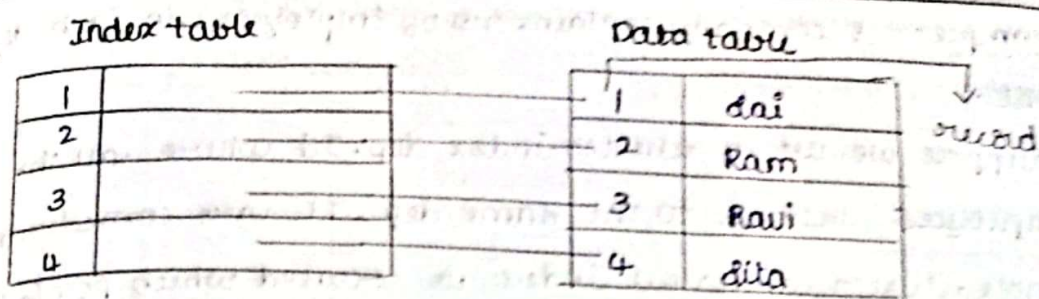
Primary index :- If the index is created based on the primary key of a table then it is known as primary indexing

- These primary keys are used to identify the records uniquely.
- These primary keys are stored in sorted order so searching is done efficiently.

A primary key index can be divided into two types i.e. dense index & sparse index

Dense index :- The dense index contains an index record for every search key value i.e. index is created for every search key value.

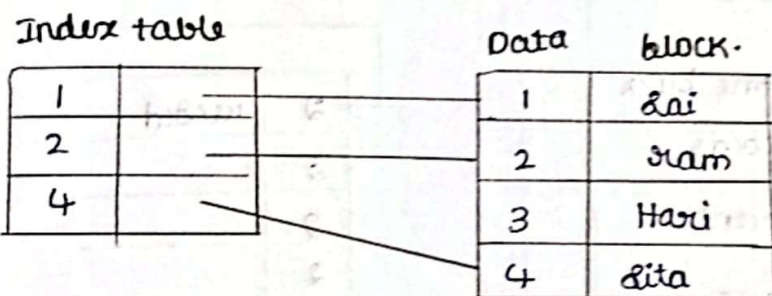
In dense index no. of records in the index table same as no. of records in the main table.



Sparse index:- The sparse index contains an index record for too few search key values in the data type i.e., index is not created for every search key.

In the sparse index no. of records in the index table not same as no. of records in the main table.

In this method index points to the records in the main table in a gap.

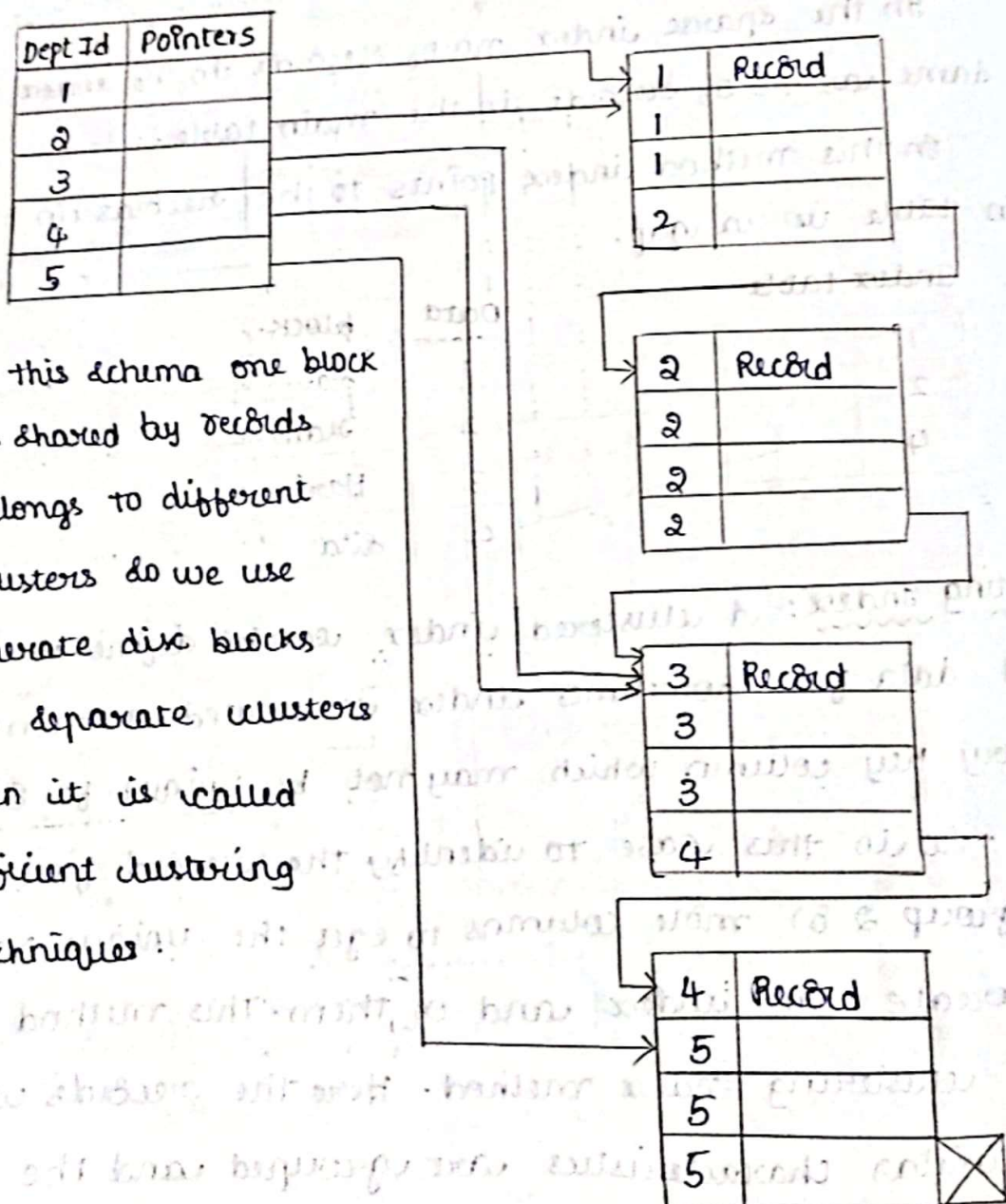


Clustering Index:- A clustered index can be defined as an ordered data file. Sometimes index is created on non-primary key column which may not be unique for each record. So in this case to identify the record fast we will group 2 or more columns to get the unique value and create an index on them. This method is called clustering index method. Here the records which have similar characteristics are grouped and the indexes are created for these groups.

Example: A company contains no. of Employees in each department.

Suppose we use a cluster index dep-Id where all the employees belongs to the same dep-Id are considered as single cluster and new index is created which points to that particular department.

Here, dep-Id is non-unique key.



In this schema one block is shared by records belongs to different clusters so we use separate disc blocks for separate clusters then it is called efficient clustering techniques!

Department

Dept_ID	Pointer
1	
2	
3	
4	
5	

1	RecBrd
1	
1	
1	

2	RecBrd
2	
2	
2	
2	

2	RecBrd
1	

3	RecBrd
3	
3	
3	

4	RecBrd
4	
4	

5	RecBrd
5	
5	
5	

Efficient clustering

Index method.

Ordered Indices:- The indices are usually added to make searching faster. The indices which are added are known as ordered indices.

Ex:- In Employee table there is thousands of records. If Id starts from 1, 2, 3 -- 1000. The searching of Employee is easy.

To search for the Employee 5 by using index (Ordered index) we can find and retrieve the Employee 5 details easily.

Employee

Id	Name	Address
1	Ram	TTD
2	Sita	RJY
3	Krishna	KKD
4	Lakshmi	MTM
5	Sai	RJY
6	Mari	KKD
⋮	⋮	⋮
1000	Ravi	TTD

Ordered Indices.

Ordered Indices Structure.

By using Ordered Index file we can retrieve the Employee title details.

5	Sai	RJY
---	-----	-----

Multi-level indexing :-

A multi level index has index file and it will refer to the first level of a multi level index. In a single level index we can create a primary index file itself.

In this case the original index file is called first level index.

Index to index file is called second level index like this we can create 3rd, 4th and so on.

By using multi level index searching is very efficient.

There are 2 types of multi level index file organization.

1. B tree file organization

2. B⁺ tree file organization.

011

111

010

001

101

000

100

010

110

000

008

018

068

005

015

065

002

012

062

016

116

006

106

006

106

006

106

006

016

116

006

106

006

106

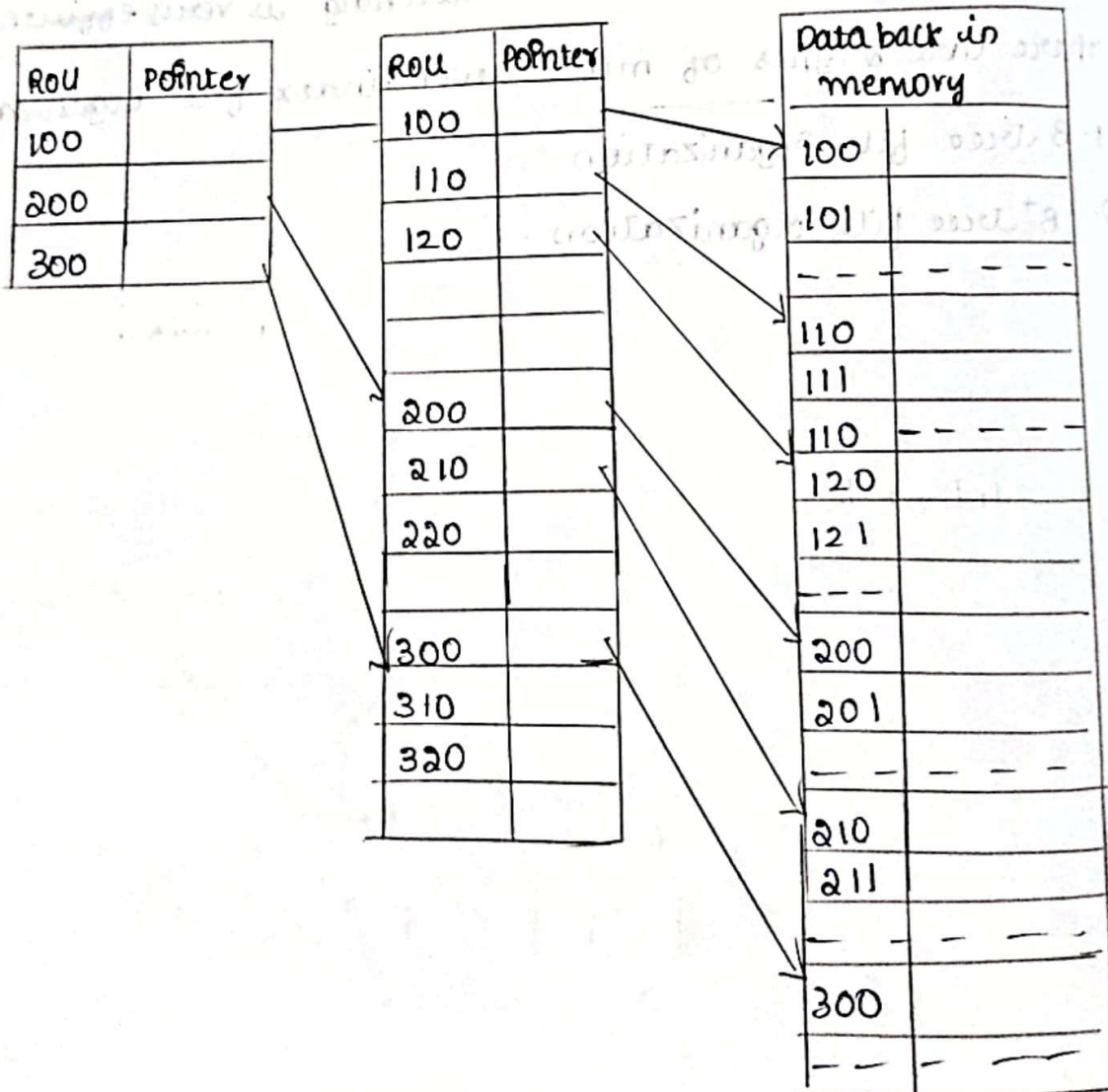
006

106

006

Secondary Index: To overcome the problem of sparse index (not mapping) secondary index is introduced in secondary index to reduce the size of mapping another level of indexing is introduced in this method initially huge range of value were deleted for the mapping in the first level. After each range of values were formed further divided into smaller ranges.

Now, the mapping done is secondary level and data is stored in secondary memory.



* B⁺ trees indexing :- B⁺ tree is a m way search tree. B⁺ trees having variation with the B⁺ tree. In B⁺ tree data pointers are stored only at leaf nodes. Leaf nodes in the B⁺ trees are linked with together. B⁺ tree having redundant nodes.

B⁺ tree operations :-

- 1) Insertion
- 2) Deletion
- 3) Search.

→ Insertion :- 1, 3, 5, 7, 9, 2, 4, 6, 8

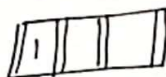
order = 4 ; m = 4 ; max children = m = 4

min children = $m/2 = 2$

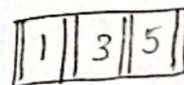
max keys = $m-1 = 4-1 = 3$

min keys = $\lceil \frac{m}{2} \rceil - 1 = 1$

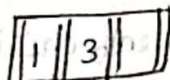
Insert - 1



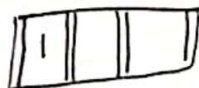
Insert - 5



Insert - 3

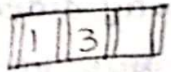


Step :- 1 Insert 1 ; Key = 1 is inserted in the first key position of the leaf node.

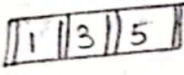


Step :- 2 Insert 3 ; Key = 3 is inserted in the second key position of the leaf node. i.e.,

K = 3 is inserted after K = 1

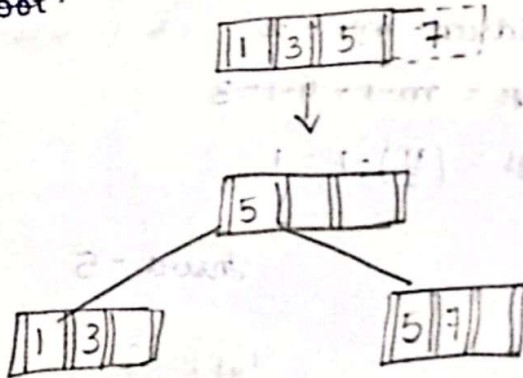


Step:-3 Insert 5; Key=5 is inserted in the third key position of the leaf node. i.e.,

K=5 is inserted after K=3. 

Step:-4 Insert 7. As the B+ tree K=7 is inserted after K=5. But leaf node is already full. Then move the $m/2$ node or middle node one level up i.e.,

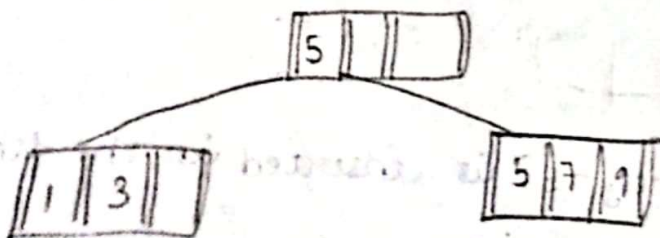
K=5 will be moved to one level up and becomes the root.



Step:-5 Insert 9

Key=9 is inserted at the rightmost leaf child of right subtree. So there is a sufficient space for K=9

i.e., K=9 is inserted after K=7.



Step

Key

Leaf

i.e.

Key

St

K

&

1

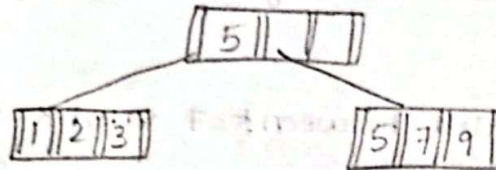
;

Step: 6 Insert 2

key = 2 is inserted in the left most leaf node of the left sub tree.

i.e;

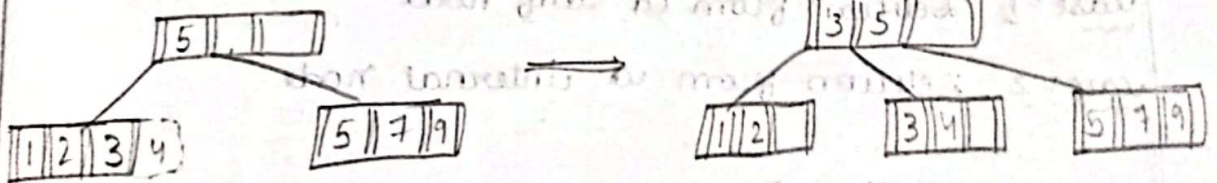
key = 2 is inserted in between $K = 1$ & 3



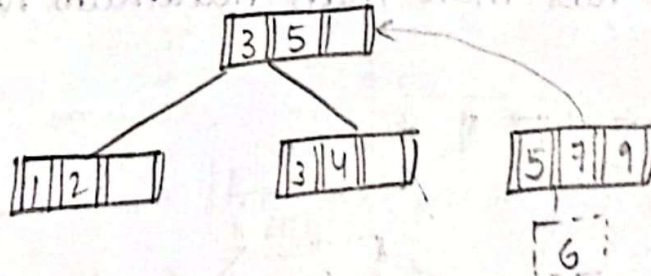
Step: 7 Insert 4

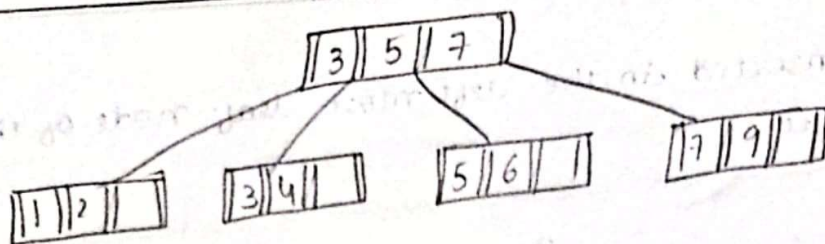
$K = 4$ is inserted in the leaf most leaf node of the left subtree. But node is already full. so we need to move $(m/2)$ node i.e. $m = 3$ one level up.

i.e. $K = 3$ will move to one level up and is inserted before $K = 5$



Step: 8 Insert 6

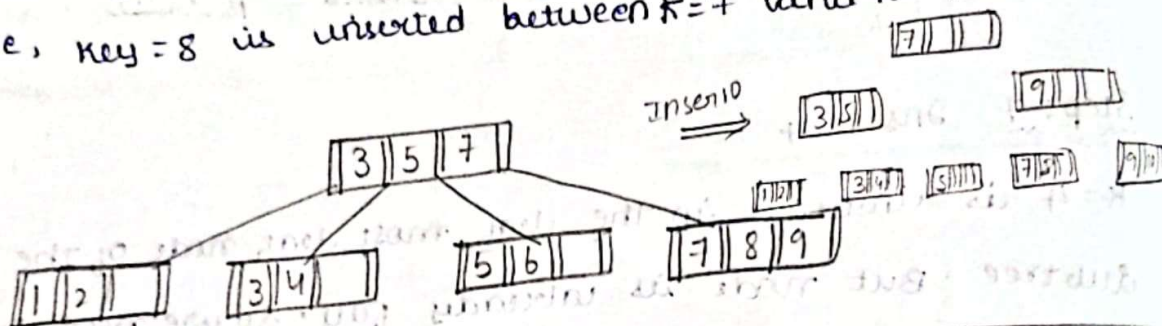




Step: 9 Insert 8.

Key = 8 is inserted at the right node leaf node of the right subtree

i.e., key = 8 is inserted between $K=7$ and $K=9$



* Deletion from a B+ tree:-

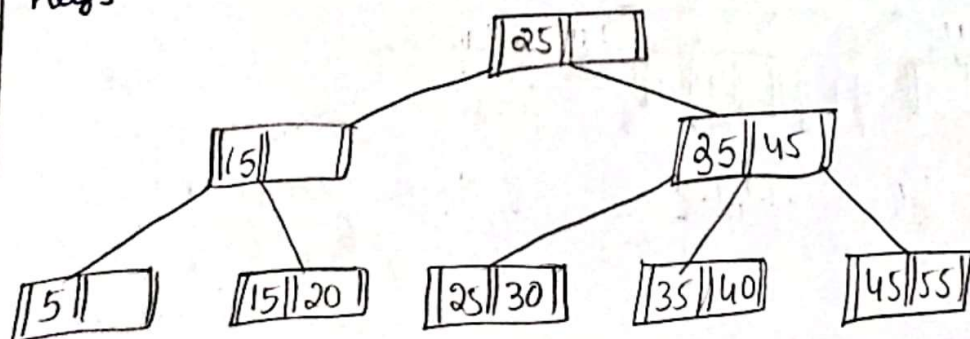
Deletion of a node from a B+ tree having 2 cases.

Case:-1 Deletion from a leaf node

Case:-2 Deletion from an internal node.

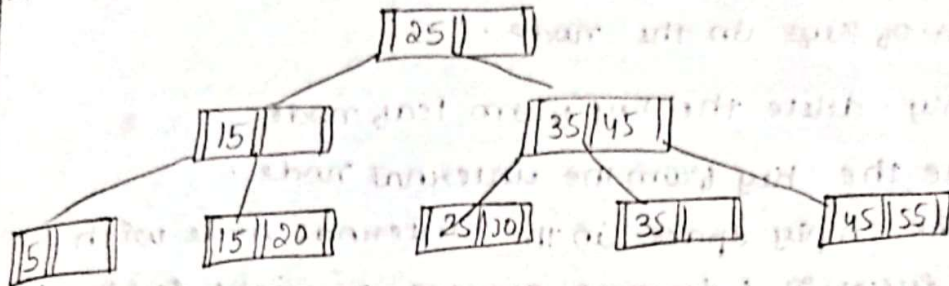
Case:-1 DELETION FROM A LEAF NODE:-

Case:-1.1 Node has more than minimum number of keys.



If the node having more than minimum no. of keys then simply delete the key.

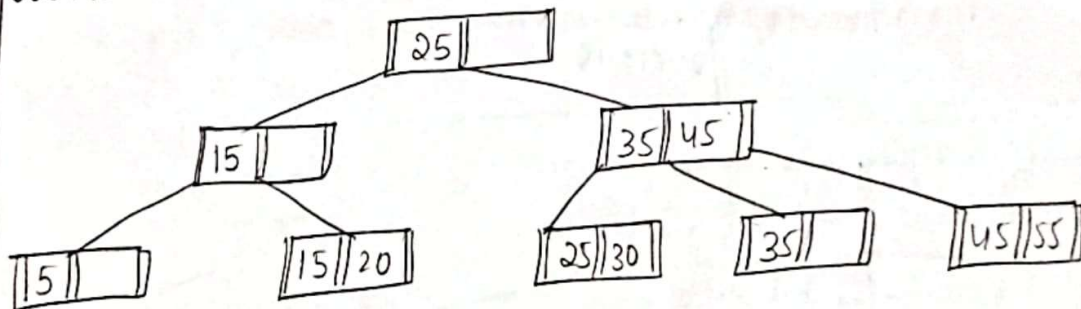
Delete 40



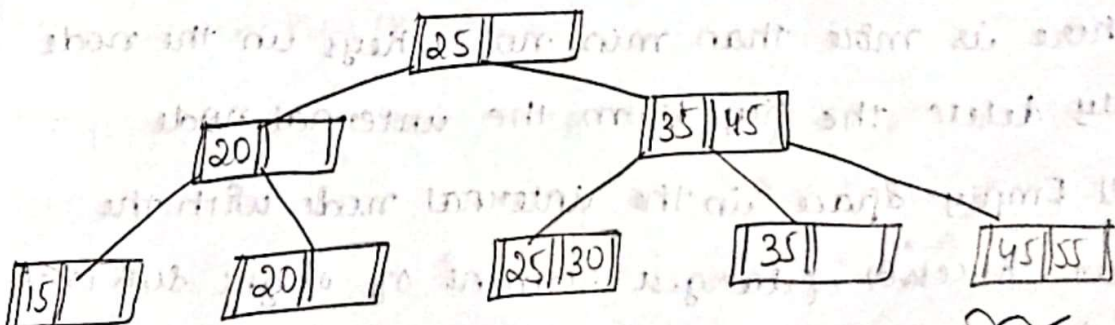
Case: 1-2 Node contains Exactly min no. of keys.

Delete the key and borrow a key from immediate sibling and add the median key of sibling to the parent.

Delete 5



Deleting 5



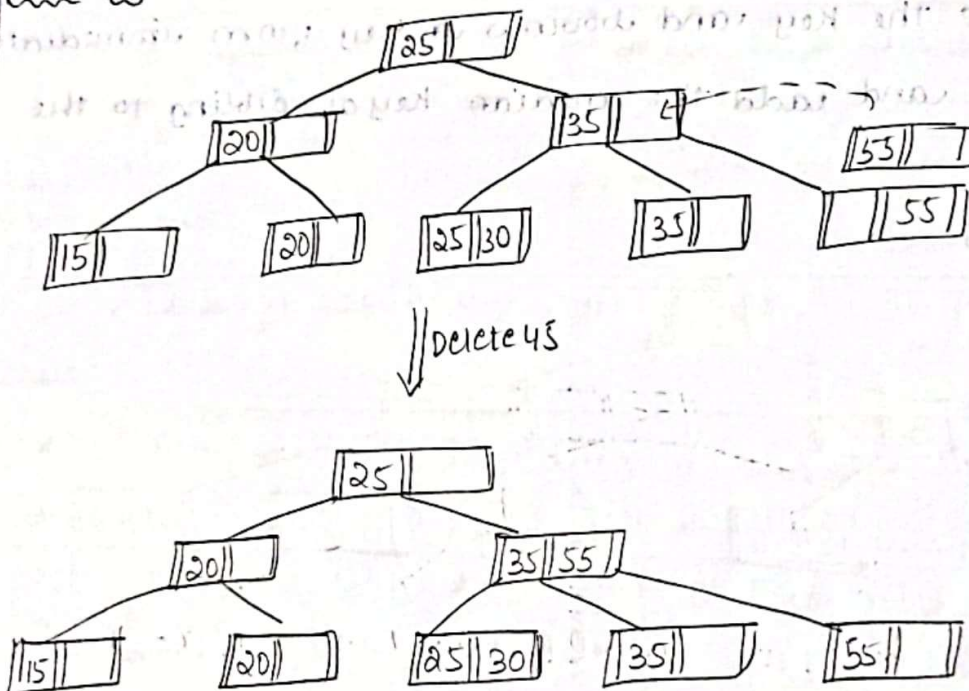
more than min keys

Case: 2 * DELETION OF A NODE FROM AN INTERNAL NODE

The key is to be deleted present in the internal node and also leaf node. If there is a node than min no. of keys in the node.

- ⇒ simply delete the key from leaf node
- ⇒ Delete the key from the internal node.
- ⇒ Fill the empty space in the internal node with in-order successor (largest element of right subtree)

Delete 45



If there is more than min no. of keys in the node simply delete the key from the internal node.

Fill Empty space in the internal node with the in-order successor (largest element of right subtree node)

* B⁺ tree

Alge

step:

If

left

step

the

step

rec

step

rec

ex

is

[15]

the

⇒ we

is

⇒ N

MODE

* B⁺ tree - search operation:-

Algorithm:-

Step:-1 compare one search key with root node keys
If search key < keys of root then traverse to the left sub tree.

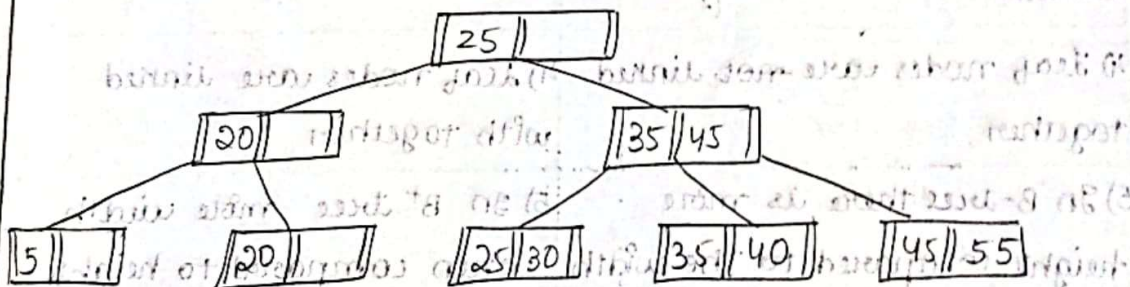
Step:-2 If search key > keys of root then traverse the right sub tree

Step:-3 Repeat steps two & three until leaf node is reached.

Step:-4 If the search key is found at leaf node then returns found key.

else,

return element not found, search key = 5



here search key (K) = 5

⇒ compare K with root node, then $K <$ root node, traverse left sub-tree

⇒ now compare K with root node in left subtree here
 $K <$ root node, then traverse left subtree of root node

→ Now, compare K with root node
Here,

$K = \text{root node}$

hence return root node i.e., $K=5$

* Difference between B-trees and B⁺ trees:

B-trees	B ⁺ -trees
1) It is a M-way search tree	1) It is a M-way search tree
2) In a B-tree keys and records can be stored in internal node as well as leaf node	2) In B ⁺ trees records are stored in leaf nodes and keys are stored in internal nodes.
3) Keys are not repeatedly stored	3) Keys are repeatedly stored
4) Leaf nodes are not linked together	4) Leaf nodes are linked with together
5) In B-tree there is more height compared to the width	5) In B ⁺ tree more width when compared to height because all leaf nodes are same distance with root
6) Searching is very slow because records are stored in leaf nodes & internal nodes	6) Searching is very fast because leaf nodes are linked with internal nodes with pointers

*

7) Sequential access of record is not possible

8) Example: Order $m=3$

max children = $m = 3$

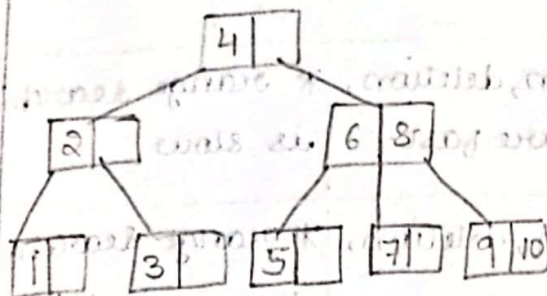
min children = $\frac{m}{2} = \frac{3}{2} = 2$
(ceil value)

max keys = $m-1 = 3-1 = 2$

min keys = $(\frac{m}{2})-1 = 2-1 = 1$

1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Order = 3



7) Sequential access of records is done very fast.

8) Example: $m=3$ 4 order

max children = $m = 4$

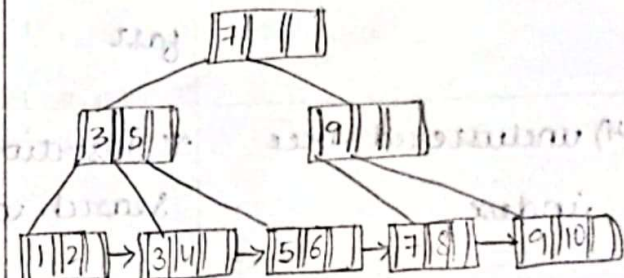
min children = $\frac{m}{2} = 2$

max keys = $m-1 = 3$

min keys = $2-1 = 1$

1, 3, 5, 7, 9, 2, 4, 6, 8, 10

Order = 4



* Comparison of file Organisation indexing and performance tuning :-

File Organisation	Advantage	disadvantage
1) Heap file	* Good storage * Insertion fast	* search slow * deletion slow
2) sorted file	* Good storage * search is faster than Heap	* Insertion slow * deletion slow.
3) clustered file	* Good storage * search, insertion and deletion are fast	* More space
4) unclustered tree index	* Insertion, deletion, search are fast.	* range searching is slow
5) unclustered hash index	* Insertion, deletion, search are fast	* range search does not exist